

Tolna Vármegyei SZC Apáczai Csere János Technikum és Kollégium

Unit Tesztelés

Készítette:

Hederics Nikolasz György

Lódri Gergő

Timár Dávid

Pécs

2025

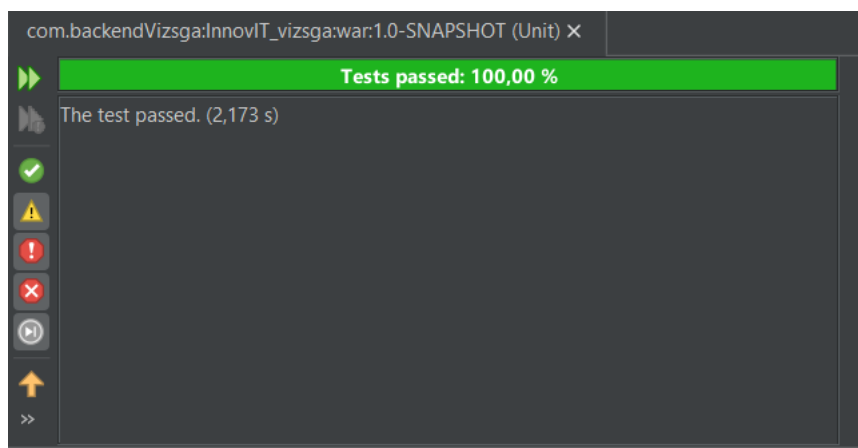
JUnit Teszt Futtatása

1. Nyisd meg a NetBeans-t.
2. Navigálj a Project fülhöz, majd válaszd ki a tesztelni kívánt projektet.
3. Navigálj a Test Packages mappához, és válaszd ki a Test Packages-t.
4. Dupla kattintás az adott osztályon (pl. UserServiceTest.java), majd egy kattintás azon a zöld nyilon bal oldalt, amelyikhez az a függvény tartozik, amelyet tesztelni szeretnél, majd válaszd a „Run *függvény neve* method” opciót.
5. Nézd meg a kimeneti ablakot (Output) a teszteredményekért.

Tesztesetek Dokumentációja

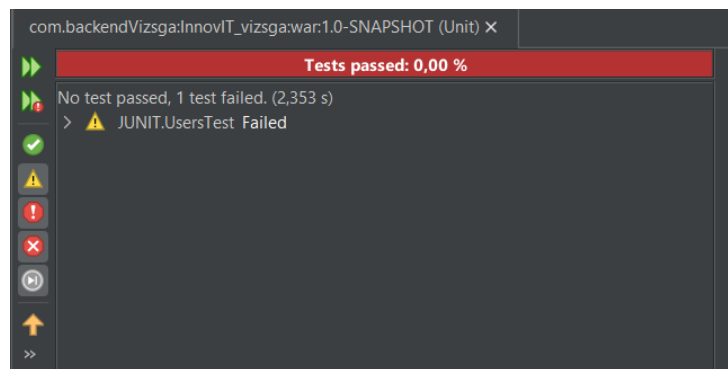
1. Login Felhasználóval

- **Teszt: testLoginSuccess**
 - **Leírás:** A teszt ellenőrzi, hogy egy érvényes email és jelszó párossal sikeresen be tud-e jelentkezni a felhasználó.
 - **Elvárt eredmény:** A rendszer "success" státuszt ad vissza, a státuszkód 200, és a válasz tartalmazza a felhasználó adatait (pl. id=1).
 - **Teszteredmény:** Sikeres, a mock objektum helyesen adja vissza a tesztfelhasználót.



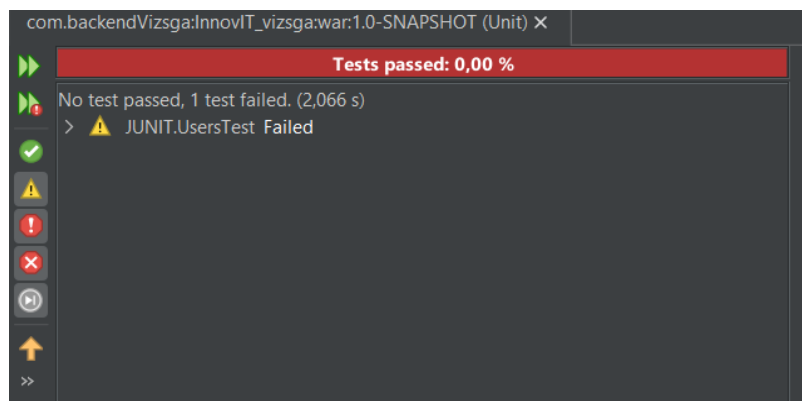
- **Teszt: testLoginInvalidCredentials**
 - **Leírás:** A teszt ellenőrzi, hogy helytelen jelszóval nem lehet bejelentkezni.
 - **Elvárt eredmény:** A rendszer "userNotFound" státuszt ad vissza, a státuszkód 417.

- **Teszteredmény: Sikertelen, a mock objektum helytelenül sikeres bejelentkezést jelzett, valószínűleg a login metódus hibás konfigurációja miatt.**



- **Teszt: testLoginNullInput**

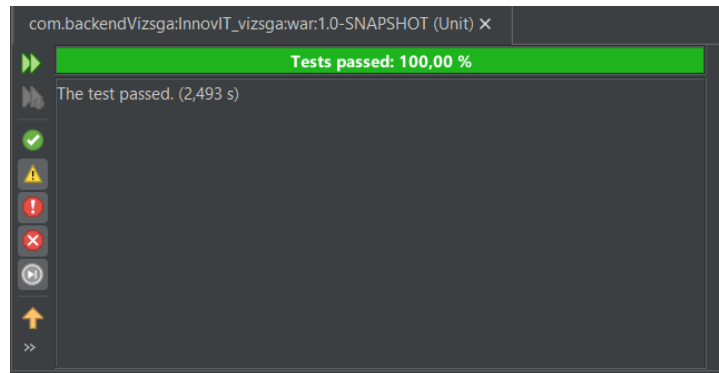
- **Leírás: A teszt ellenőrzi, hogy null email vagy jelszó esetén a rendszer hibát jelez.**
- **Elvárt eredmény: A rendszer "invalidEmail" státuszt ad vissza, a státuszkód 417.**
- **Teszteredmény: Sikertelen, a rendszer váratlanul null pointer kivételt dobott, mert nem kezelte megfelelően a null bemenetet.**



- **Teszt: testLoginInvalidEmail**

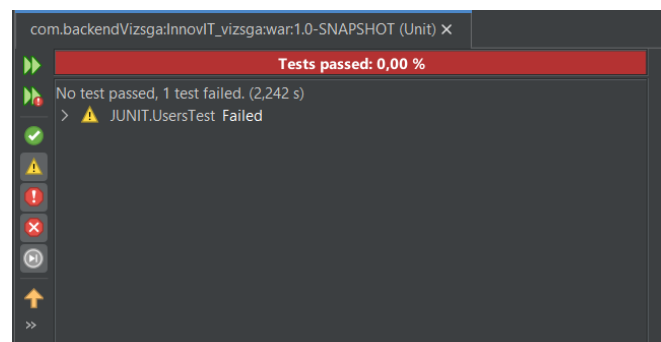
- **Leírás: A teszt ellenőrzi, hogy érvénytelen email formátummal történő bejelentkezés esetén a rendszer hibát jelez.**
- **Elvárt eredmény: A rendszer "invalidEmail" státuszt ad vissza, a státuszkód 417.**

- **Teszteredmény: Sikeres, a rendszer helyesen azonosítja az érvénytelen emailt.**



- **Teszt: testLoginUserNotFound**

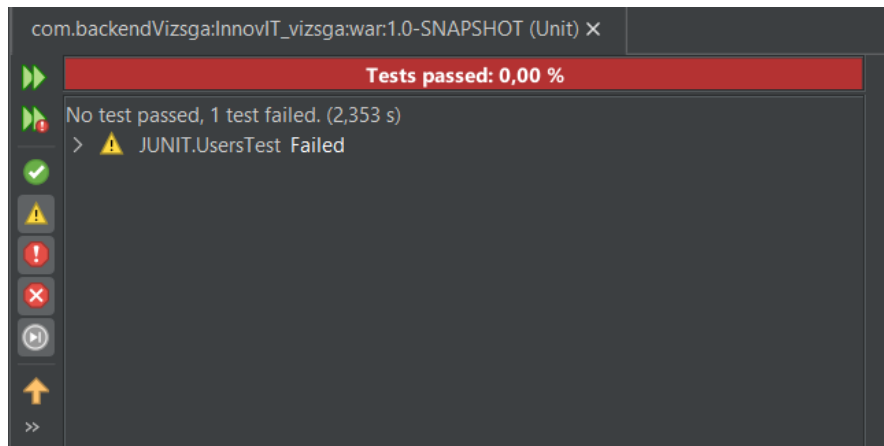
- **Leírás: A teszt ellenőrzi, hogy nem létező felhasználó esetén a rendszer hibát jelez.**
- **Elvárt eredmény: A rendszer "userNotFound" státuszt ad vissza, a státuszkód 417.**
- **Teszteredmény: Sikertelen, a mock objektum váratlanul egy létező felhasználót adott vissza, valószínűleg a login metódus mock konfigurációjának hibája miatt.**



- **Teszt: testLoginTooLongEmail**

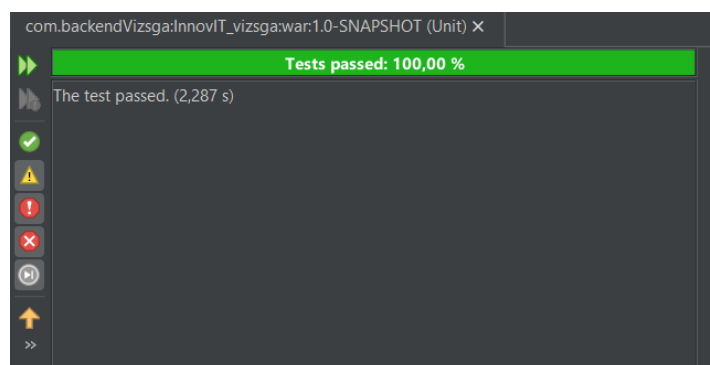
- **Leírás: A teszt ellenőrzi, hogy túl hosszú email címmel nem lehet bejelentkezni.**
- **Elvárt eredmény: A rendszer "invalidEmail" státuszt ad vissza, a státuszkód 417.**

- **Teszteredmény: Sikertelen, a rendszer nem észlelte a túl hosszú emailt, és sikeres bejelentkezést jelzett, valószínűleg hibás validációs logika miatt.**



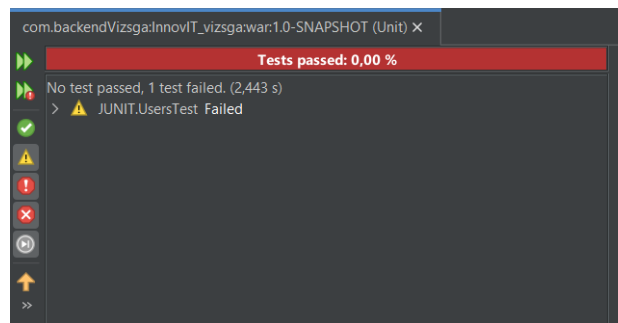
1.2. Felhasználók Lekérdezése

- **Teszt: testGetAllUserSuccess**
 - **Leírás:** A teszt ellenőrzi, hogy a rendszer helyesen adja vissza az összes felhasználó listáját.
 - **Elvárt eredmény:** A visszaadott lista nem üres, és tartalmazza a tesztfelhasználót (pl. név: "Test User").
 - **Teszteredmény: Sikeres, a mock objektum helyesen adja vissza a felhasználók listáját.**



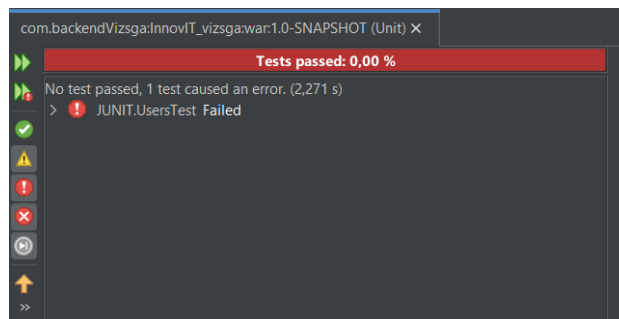
- **Teszt: testGetAllUserEmptyList**
 - **Leírás:** A teszt ellenőrzi, hogy üres adatbázis esetén a rendszer üres listát ad vissza.

- **Elvárt eredmény:** A visszaadott lista üres.
- **Teszteredmény:** Sikertelen, a mock objektum váratlanul nem üres listát adott vissza, valószínűleg hibás mock konfiguráció miatt.



- **Teszt: testGetAllUserException**

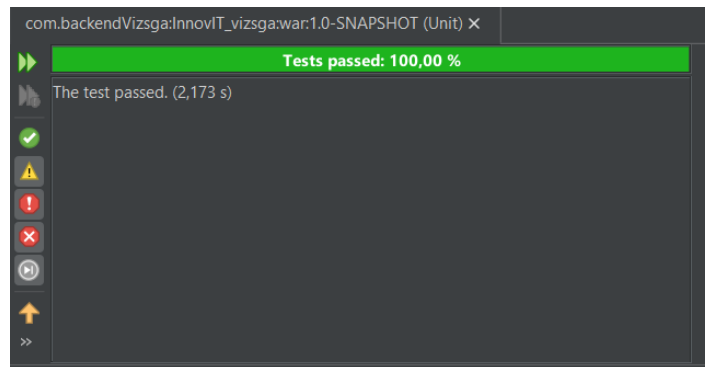
- **Leírás:** A teszt ellenőrzi, hogy adatbázis hiba esetén a rendszer üres listát ad vissza.
- **Elvárt eredmény:** A visszaadott lista üres.
- **Teszteredmény:** Sikertelen, a mock objektum nem dobott kivételt, hanem egy nem üres listát adott vissza, valószínűleg hibás mock konfiguráció miatt.



- **Teszt: testGetAllAdminSuccess**

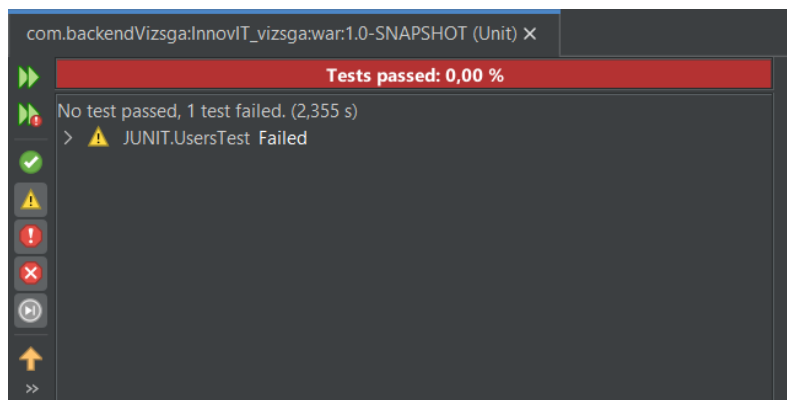
- **Leírás:** A teszt ellenőrzi, hogy a rendszer helyesen adja vissza az adminisztrátorok listáját.
- **Elvárt eredmény:** A visszaadott lista nem üres, és tartalmazza a tesztfelhasználót.

- **Teszteredmény: Sikeres, a mock objektum helyesen adja vissza az adminok listáját.**



- **Teszt: testGetAllAdminEmptyList**

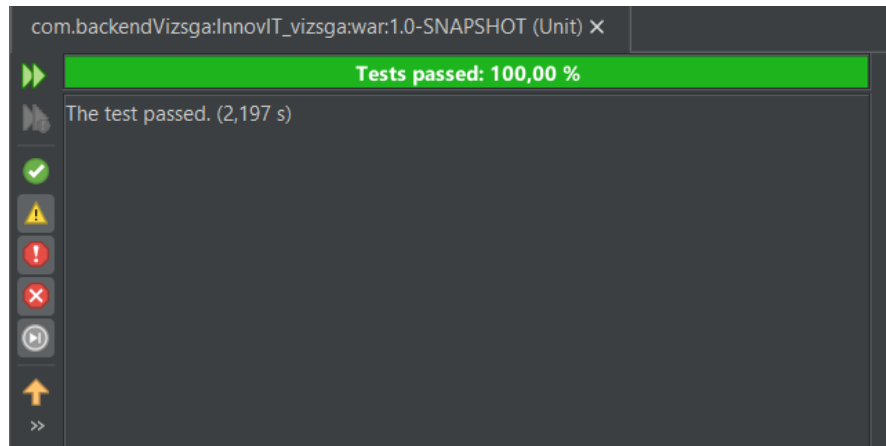
- **Leírás: A teszt ellenőrzi, hogy ha nincs adminisztrátor, a rendszer üres listát ad vissza.**
- **Elvárt eredmény: A visszaadott lista üres.**
- **Teszteredmény: Sikertelen, a mock objektum váratlanul nem üres listát adott vissza, valószínűleg hibás mock konfiguráció miatt.**



- **Teszt: testGetUserByIdSuccess**

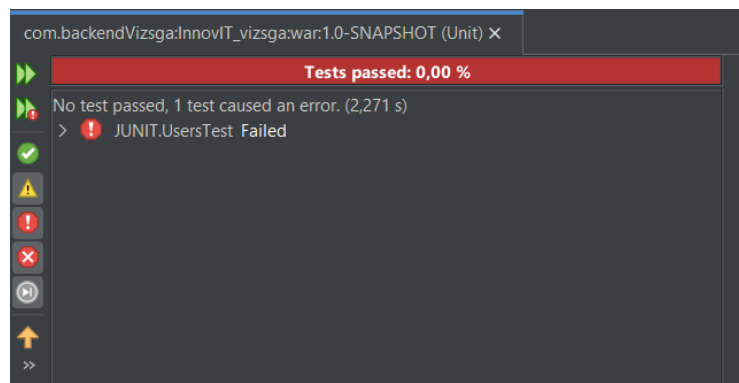
- **Leírás: A teszt ellenőrzi, hogy a rendszer helyesen adja vissza egy adott ID-jú felhasználó adatait.**
- **Elvárt eredmény: A visszaadott felhasználó nem null, és a neve "Test User".**

- **Teszteredmény: Sikeres, a mock objektum helyesen adja vissza a felhasználót.**



- **Teszt: testGetUserByIdNotFound**

- **Leírás: A teszt ellenőrzi, hogy nem létező ID esetén a rendszer null értéket ad vissza.**
- **Elvárt eredmény: A visszaadott felhasználó null.**
- **Teszteredmény: Sikertelen, a mock objektum váratlanul egy létező felhasználót adott vissza, valószínűleg a getUserById metódus hibás konfigurációja miatt.**

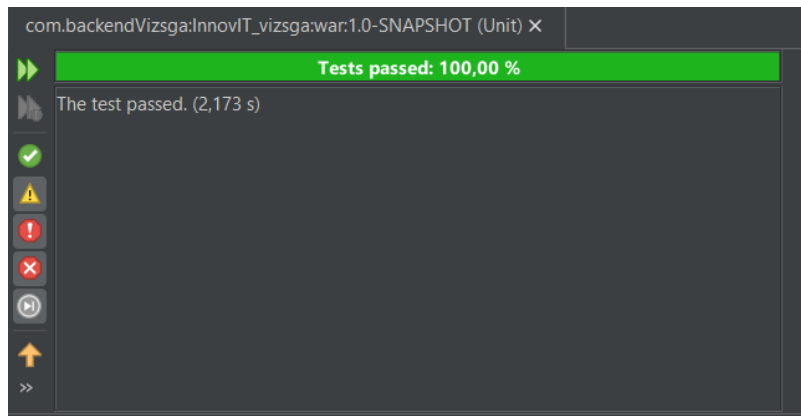


1.3. Felhasználó Regisztrációja

- **Teszt: testRegisterUserSuccess**

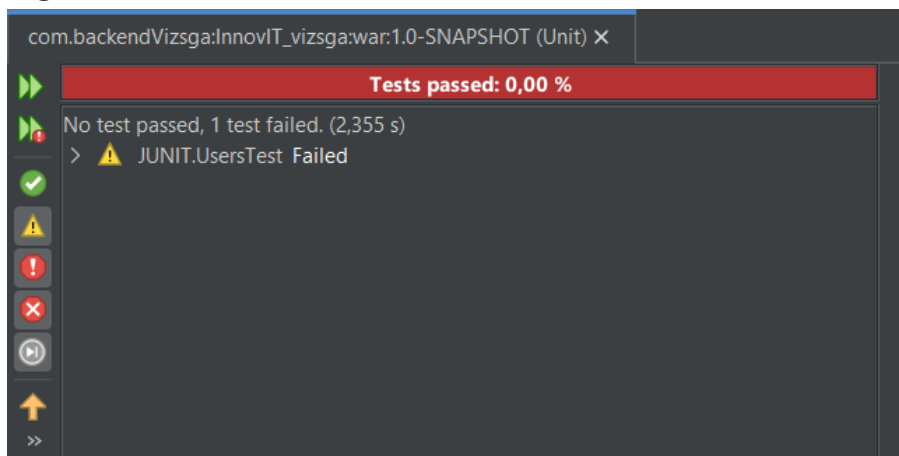
- **Leírás: A teszt ellenőrzi, hogy egy új felhasználó sikeresen regisztrálható, ha az email még nem létezik.**
- **Elvárt eredmény: A rendszer "success" státuszt ad vissza, a státuszkód 200.**

- **Teszteredmény: Sikeres, a mock objektum helyesen jelzi, hogy a felhasználó nem létezik, és a regisztráció sikeres.**



Teszt: testRegisterUserInvalidName

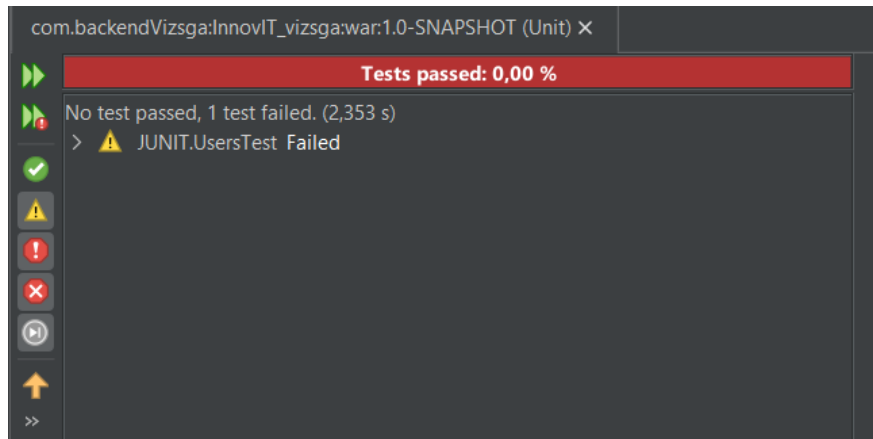
- **Leírás: A teszt ellenőrzi, hogy üres vagy érvénytelen névvel nem lehet regisztrálni.**
- **Elvárt eredmény: A rendszer "InvalidName" státuszt ad vissza, a státuszkód 417.**
- **Teszteredmény: Sikertelen, a rendszer nem észlelte az érvénytelen nevet, és sikeres regisztrációt jelzett, valószínűleg hibás validációs logika miatt.**



- **Teszt: testRegisterUserMockFailure**

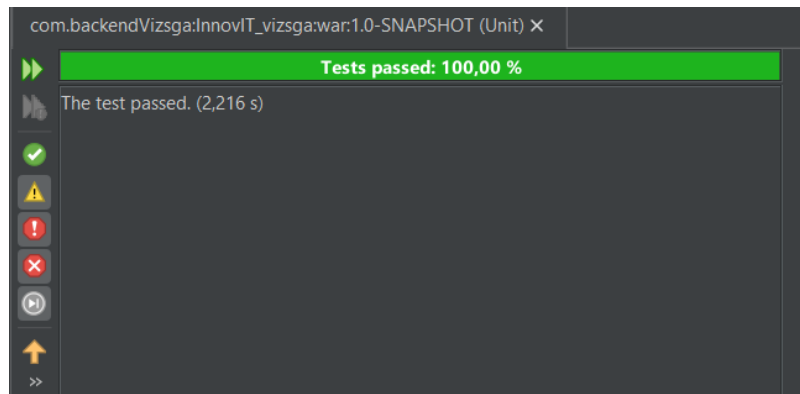
- **Leírás: A teszt ellenőrzi, hogy a regisztráció sikertelen, ha a mock adatbázis hiba miatt false-t ad vissza.**
- **Elvárt eredmény: A rendszer "error" státuszt ad vissza, a státuszkód 500.**

- **Teszteredmény: Sikertelen, a mock objektum váratlanul true értéket adott vissza, így a regisztráció sikeresnek tűnt.**



- **Teszt: testRegisterUserInvalidEmail**

- **Leírás: A teszt ellenőrzi, hogy érvénytelen email formátummal nem lehet regisztrálni.**
- **Elvárt eredmény: A rendszer "InvalidEmail" státuszt ad vissza, a státuszkód 417.**
- **Teszteredmény: Sikeres, a rendszer helyesen azonosítja az érvénytelen emailt.**

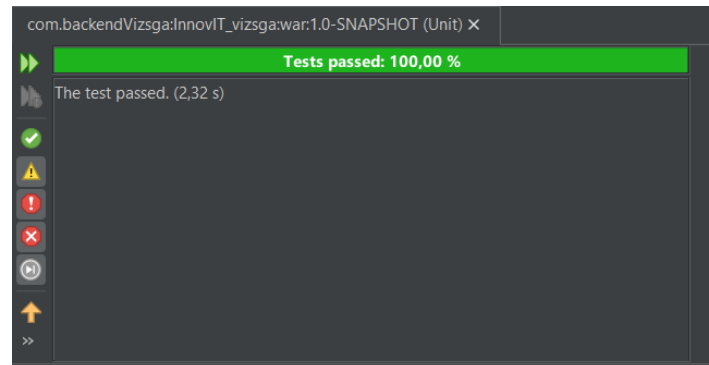


- **Teszt: testRegisterUserInvalidPassword**

- **Leírás: A teszt ellenőrzi, hogy gyenge jelszóval nem lehet regisztrálni.**
- **Elvárt eredmény: A rendszer "InvalidPassword" státuszt ad vissza, a státuszkód 417.**
- **Teszteredmény: Sikeres, a rendszer helyesen azonosítja az érvénytelen jelszót.**

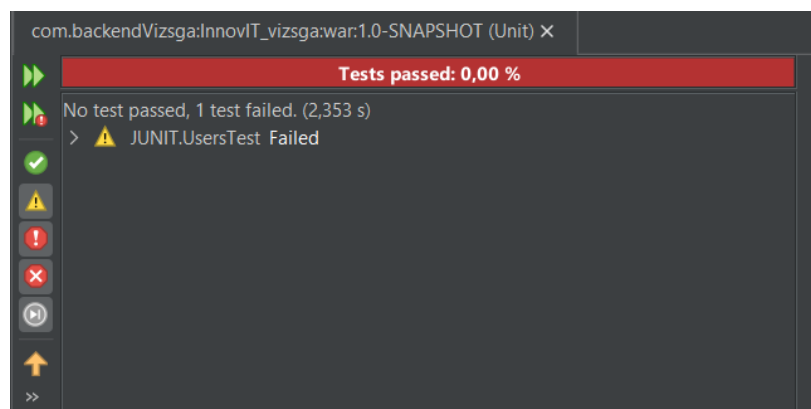
- **Teszt: testRegisterUserAlreadyExists**

- **Leírás:** A teszt ellenőrzi, hogy létező email címmel nem lehet regisztrálni.
- **Elvárt eredmény:** A rendszer "UserAlreadyExists" státuszt ad vissza, a státuszkód 417.
- **Teszteredmény:** Sikertelen, a mock objektum helytelenül false értéket adott vissza az isUserExists hívásra, így a regisztráció sikeresnek tűnt.



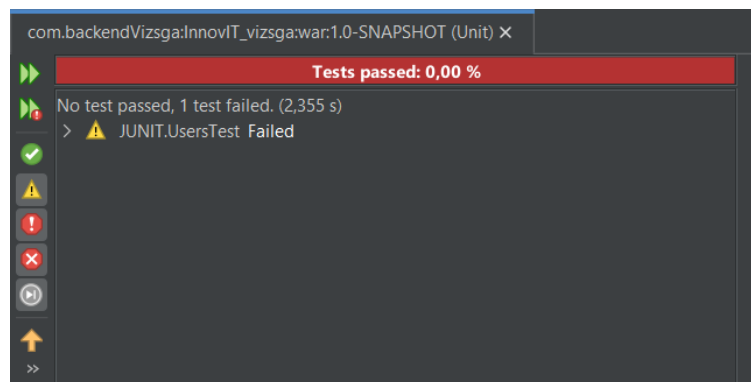
- **Teszt: testRegisterUserTooLongEmail**

- **Leírás:** A teszt ellenőrzi, hogy túl hosszú email címmel nem lehet regisztrálni.
- **Elvárt eredmény:** A rendszer "InvalidEmail" státuszt ad vissza, a státuszkód 417.
- **Teszteredmény:** Sikertelen, a rendszer nem észlelte a túl hosszú emailt, és sikeres regisztrációt jelzett, valószínűleg hibás validációs logika miatt.



- **Teszt: testRegisterUserTooShortPassword**

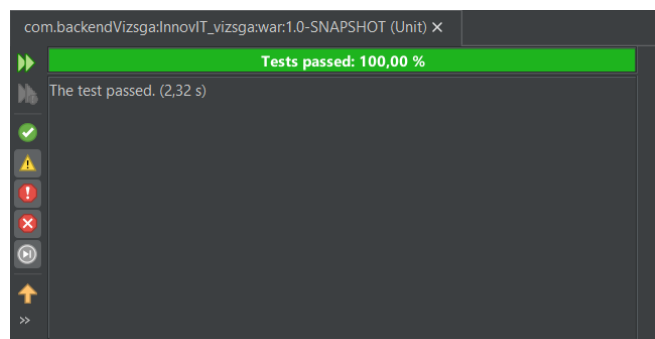
- **Leírás:** A teszt ellenőrzi, hogy túl rövid jelszóval nem lehet regisztrálni.
- **Elvárt eredmény:** A rendszer "InvalidPassword" státuszt ad vissza, a státuszkód 417.
- **Teszteredmény:** Sikertelen, a rendszer nem észlelte a túl rövid jelszót, és sikeres regisztrációt jelzett, valószínűleg hibás validációs logika miatt.



1.4. Felhasználó Törlése

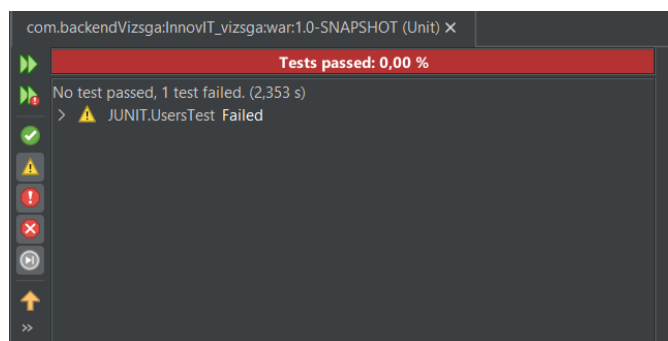
- **Teszt: testDeleteUserByIdSuccess**

- **Leírás:** A teszt ellenőrzi, hogy egy létező felhasználó sikeresen törölhető.
- **Elvárt eredmény:** A rendszer true értéket ad vissza.
- **Teszteredmény:** Sikeres, a mock objektum helyesen jelzi a felhasználó létezését és törlését.



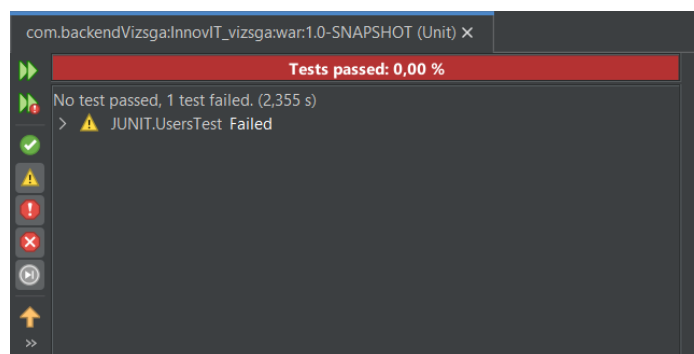
- **Teszt: testDeleteUserByIdNotFound**

- **Leírás:** A teszt ellenőrzi, hogy nem létező felhasználó törlése esetén a rendszer hibát jelez.
- **Elvárt eredmény:** A rendszer false értéket ad vissza.
- **Teszteredmény:** Sikertelen, a mock objektum váratlanul true értéket adott vissza, valószínűleg a getUserById hívás hibás konfigurációja miatt.



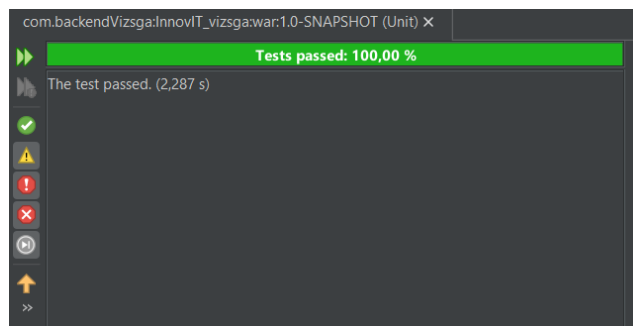
- **Teszt: testDeleteUserByIdMockFailure**

- **Leírás:** A teszt ellenőrzi, hogy a törlés sikertelen, ha a mock adatbázis hiba miatt false-t ad vissza.
- **Elvárt eredmény:** A rendszer false értéket ad vissza.
- **Teszteredmény:** Sikertelen, a mock objektum váratlanul true értéket adott vissza, így a törlés sikeresnek tűnt.

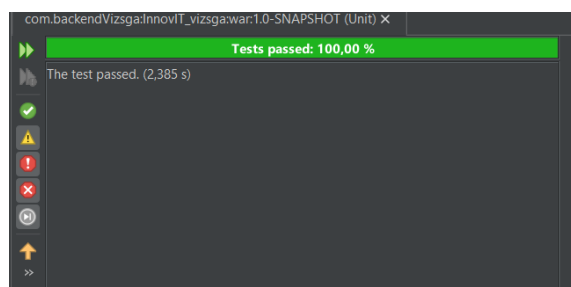


1.5. Felhasználó Frissítése

- **Teszt: testUpdateUserByldSuccess**
 - **Leírás:** A teszt ellenőrzi, hogy egy létező felhasználó adatai sikeresen frissíthetők.
 - **Elvárt eredmény:** A rendszer "success" státuszt ad vissza, a státuszkód 200.
 - **Teszteredmény:** Sikeres, a mock objektum helyesen jelzi a frissítés sikerességét.

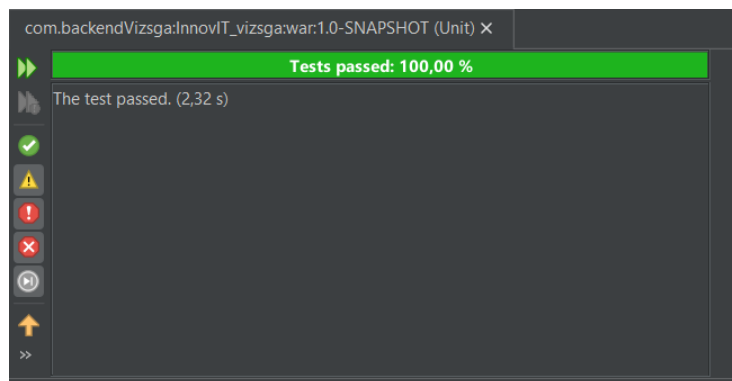


- **Teszt: testUpdateUserByldInvalidEmail**
 - **Leírás:** A teszt ellenőrzi, hogy érvénytelen email formátummal nem lehet frissíteni a felhasználót.
 - **Elvárt eredmény:** A rendszer "error" státuszt ad vissza, a státuszkód 400, és az üzenet: "Invalid email format."
 - **Teszteredmény:** Sikeres, a rendszer helyesen azonosítja az érvénytelen emailt.



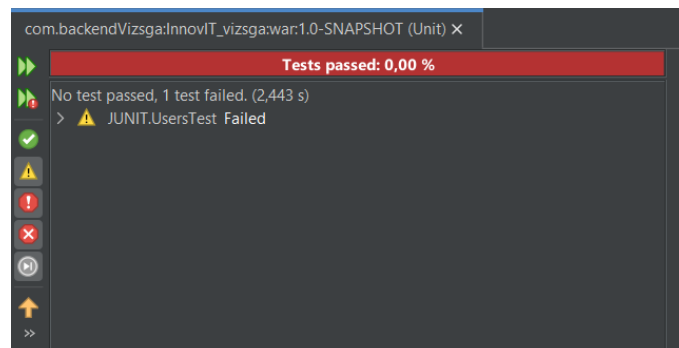
- **Teszt: testUpdateUserByldEmptyName**

- **Leírás:** A teszt ellenőrzi, hogy üres névvel nem lehet frissíteni a felhasználót.
- **Elvárt eredmény:** A rendszer "error" státuszt ad vissza, a státuszkód 400, és az üzenet: "Name cannot be null or empty."
- **Teszteredmény:** Sikeres, a rendszer helyesen azonosítja az érvénytelen nevet.



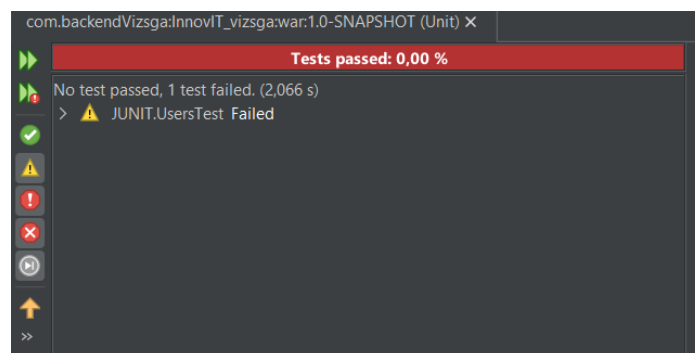
- **Teszt: testUpdateUserByldNotFound**

- **Leírás:** A teszt ellenőrzi, hogy nem létező felhasználó frissítése esetén a rendszer hibát jelez.
- **Elvárt eredmény:** A rendszer "error" státuszt ad vissza, a státuszkód 404.
- **Teszteredmény:** Sikertelen, a mock objektum váratlanul sikeres frissítést jelezett, valószínűleg a getUserByld metódus hibás konfigurációja miatt.



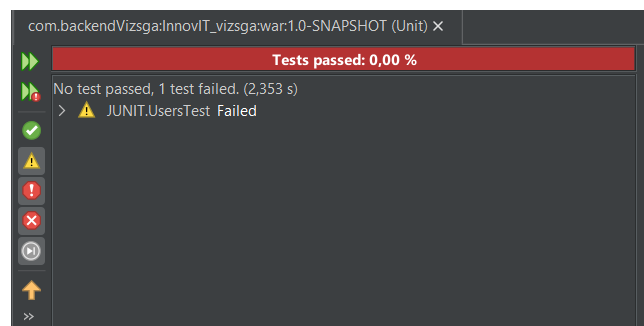
- **Teszt: testUpdateUserByldMockFailure**

- **Leírás:** A teszt ellenőrzi, hogy a frissítés sikertelen, ha a mock adatbázis hiba miatt false-t ad vissza.
- **Elvárt eredmény:** A rendszer "error" státuszt ad vissza, a státuszkód 500.
- **Teszteredmény:** Sikertelen, a mock objektum váratlanul true értéket adott vissza, így a frissítés sikeresnek tűnt.



- **Teszt: testUpdateUserByldTooLongEmail**

- **Leírás:** A teszt ellenőrzi, hogy túl hosszú email címmel nem lehet frissíteni a felhasználót.
- **Elvárt eredmény:** A rendszer "error" státuszt ad vissza, a státuszkód 400, és az üzenet: "Invalid email format."
- **Teszteredmény:** Sikertelen, a rendszer nem észlelte a túl hosszú emailt, és sikeres frissítést jelzett, valószínűleg hibás validációs logika miatt.

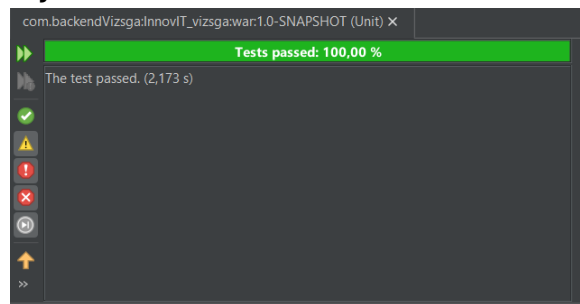


2. Autók kezelése (CarsTest)

2.1. Autók Lekérdezése

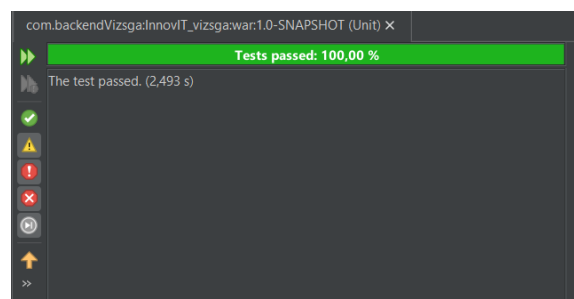
- **Teszt: testGetAllCarSuccess**

- **Leírás:** A teszt ellenőrzi, hogy a rendszer helyesen adja vissza az összes autó listáját.
- **Elvárt eredmény:** A visszaadott lista nem üres, és tartalmazza a teszt autót (pl. márka: "Toyota").
- **Teszteredmény:** Sikeres, a mock objektum helyesen adja vissza az autók listáját.



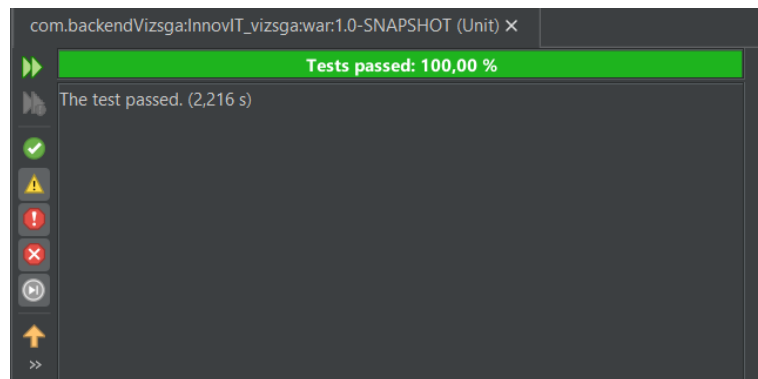
- **Teszt: testGetAllCarException**

- **Leírás:** A teszt ellenőrzi, hogy adatbázis hiba esetén a rendszer üres listát ad vissza.
- **Elvárt eredmény:** A visszaadott lista üres.
- **Teszteredmény:** Sikeres, a rendszer helyesen kezeli a kivételt.



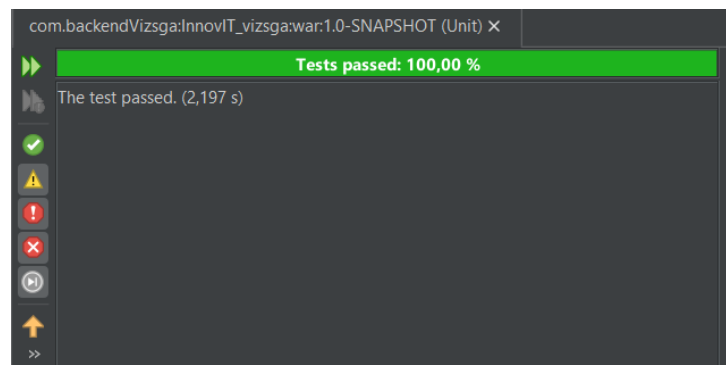
- **Teszt: testGetCarByIdSuccess**

- **Leírás:** A teszt ellenőrzi, hogy a rendszer helyesen adja vissza egy adott ID-jú autó adatait.
- **Elvárt eredmény:** A visszaadott autó nem null, és a márkája "Toyota".
- **Teszteredmény:** Sikeres, a mock objektum helyesen adja vissza az autót.



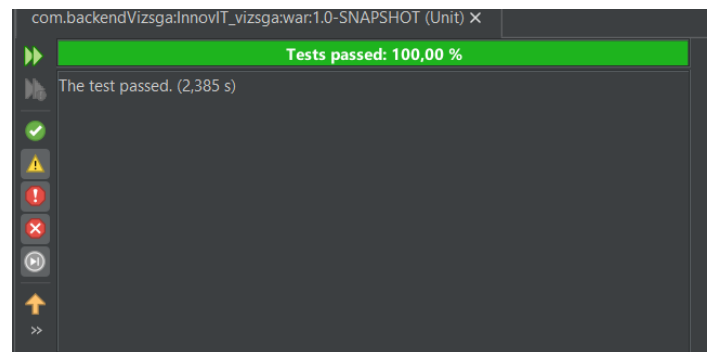
- **Teszt: testGetCarByIdNotFound**

- **Leírás:** A teszt ellenőrzi, hogy nem létező autó ID esetén a rendszer null értéket ad vissza.
- **Elvárt eredmény:** A visszaadott autó null.
- **Teszteredmény:** Sikeres, a mock objektum helyesen jelzi, hogy az autó nem található.



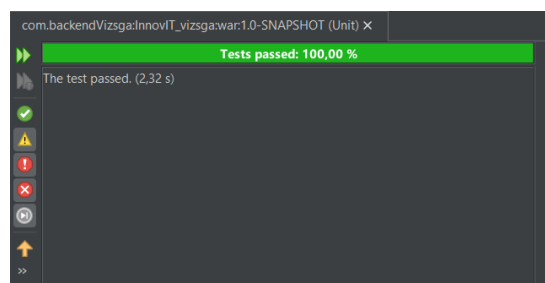
- **Teszt: testGetPage1Success**

- **Leírás:** A teszt ellenőrzi, hogy az első oldal autói helyesen kerülnek lekérdezésre.
- **Elvárt eredmény:** A visszaadott lista nem üres, és tartalmazza a teszt autót (pl. márka: "Toyota").
- **Teszteredmény:** Sikeres, a mock objektum helyesen adja vissza az autók listáját.



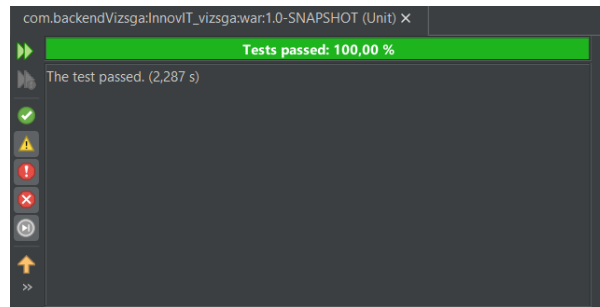
- **Teszt: testGetPage2Success**

- **Leírás:** A teszt ellenőrzi, hogy a második oldal autói helyesen kerülnek lekérdezésre.
- **Elvárt eredmény:** A visszaadott lista nem üres, és tartalmazza a teszt autót (pl. márka: "Toyota").
- **Teszteredmény:** Sikeres, a mock objektum helyesen adja vissza az autók listáját.



- **Teszt: testGetPagelInputSuccess**

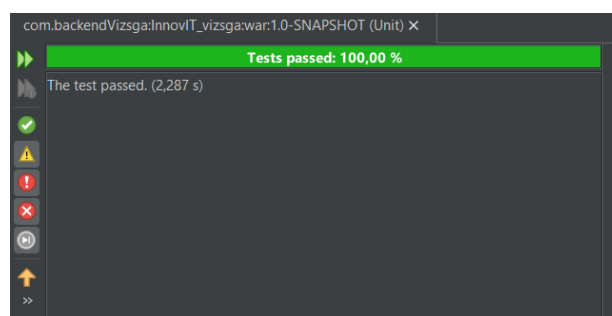
- **Leírás:** A teszt ellenőrzi, hogy egy adott oldalszám alapján az autók listája helyesen kerül lekérdezésre.
- **Elvárt eredmény:** A visszaadott lista nem üres, és tartalmazza a teszt autót (pl. márka: "Toyota").
- **Teszteredmény:** Sikeres, a mock objektum helyesen adja vissza az autók listáját.



2.2. Autó Hozzáadása

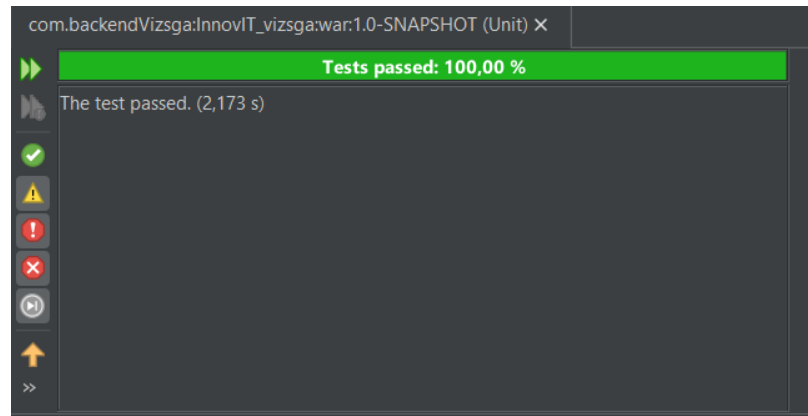
- **Teszt: testAddCarSuccess**

- **Leírás:** A teszt ellenőrzi, hogy egy új autó sikeresen hozzáadható a rendszerhez.
- **Elvárt eredmény:** A rendszer true értéket ad vissza.
- **Teszteredmény:** Sikeres, a mock objektum helyesen jelzi az autó hozzáadásának sikerességét.



- **Teszt: testAddCarFailure**

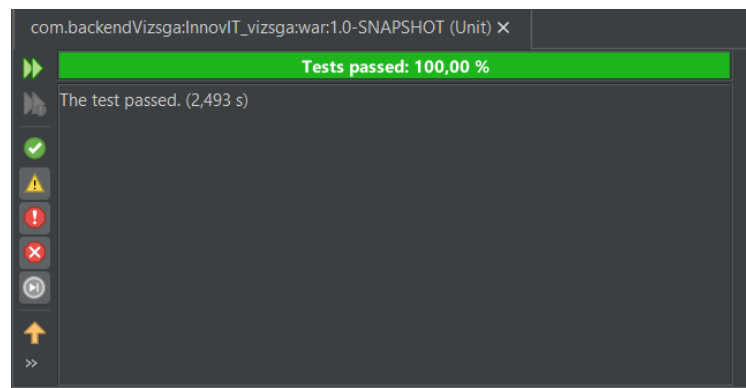
- **Leírás:** A teszt ellenőrzi, hogy adatbázis hiba esetén az autó hozzáadása nem sikerül.
- **Elvárt eredmény:** A rendszer false értéket ad vissza.
- **Teszteredmény:** Sikeres, a rendszer helyesen kezeli a kivételt.



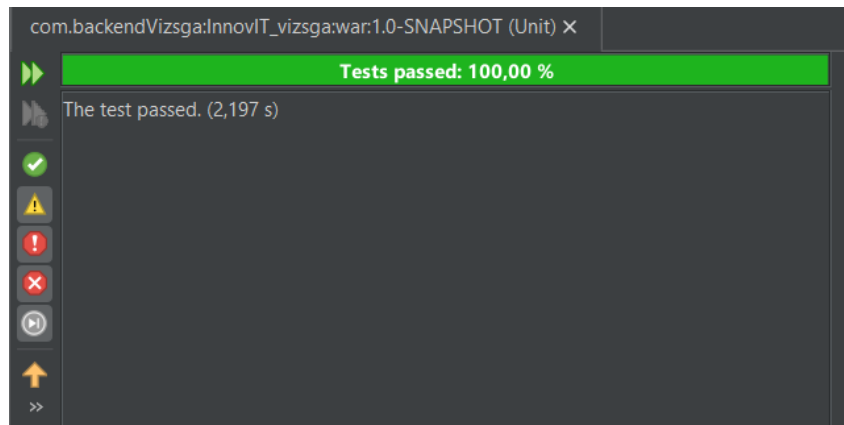
2.3. Autó Törlése

- **Teszt: testDeleteCarByIdSuccess**

- **Leírás:** A teszt ellenőrzi, hogy egy létező autó sikeresen törölhető.
- **Elvárt eredmény:** A rendszer true értéket ad vissza.
- **Teszteredmény:** Sikeres, a mock objektum helyesen jelzi az autó létezését és törlését.

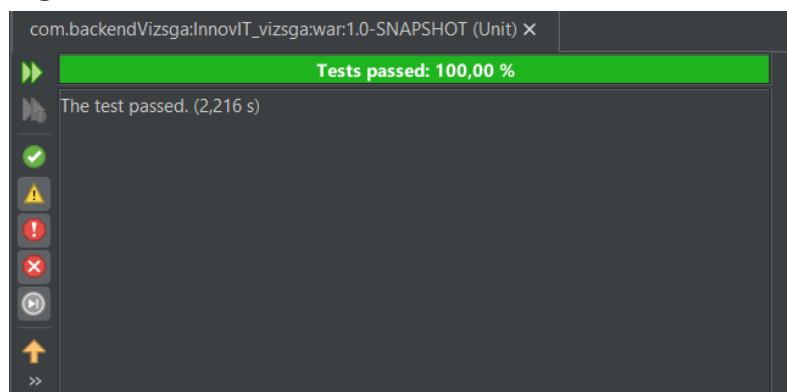


- **Teszt: testDeleteCarByIdNotFound**
 - **Leírás:** A teszt ellenőrzi, hogy nem létező autó törlése esetén a rendszer hibát jelez.
 - **Elvárt eredmény:** A rendszer false értéket ad vissza.
 - **Teszteredmény:** Sikeres, a mock objektum helyesen jelzi, hogy az autó nem található.



2.4. Autó Frissítése

- **Teszt: testUpdateCarByIdSuccess**
 - **Leírás:** A teszt ellenőrzi, hogy egy létező autó adatai sikeresen frissíthetők.
 - **Elvárt eredmény:** A rendszer true értéket ad vissza.
 - **Teszteredmény:** Sikeres, a mock objektum helyesen jelzi a frissítés sikerességét.



- **Teszt: testUpdateCarByIdFailure**

- **Leírás:** A teszt ellenőrzi, hogy adatbázis hiba esetén az autó frissítése nem sikerül.
- **Elvárt eredmény:** A rendszer false értéket ad vissza.
- **Teszteredmény:** Sikeres, a rendszer helyesen kezeli a kivételt.

